THE CHALLENGE · OPEN A PROTECTED RECORD

A plaintext flag?
Too good to be true.

SUPPLIED FILE

`beacon_flash.bin`

BOOT | PROGRAM | DATA

A copy of the memory
from an ESP32-C6 device.

→

WHAT THE PROGRAM CHECKS

Activation code?

_ _ _ _ _ _ _ _

The correct code allows
the record to be decoded.

THE TRAP ~~BA26{n0t_th15_0n3_k33p_d1gg1ng}~~ Stored in plaintext, but not the result we need.

THE TAKEAWAY

Finding text that looks like a flag **is not enough to prove the solve.**

THE BREAKTHROUGH · RECREATE THE ACTIVATION CHECK

The beacon calculates the code.
The agent repeats the calculation.

TWO READABLE STRINGS IN THE DUMP

`LilyGO T7-C6`

`BA26-BEACON-01`

→

THE PROGRAM TURNS THEM INTO TWO NUMBERS

`56ea3ff6`

`cd1014a9`

Calculated “fingerprints”.

→

COMBINED AS THE PROGRAM DOES

`B80FEF82`

The expected code.

SUPPLIED CODE `?` = ? EXPECTED CODE `B80FEF82` → **If equal: allow decoding.**

THE TAKEAWAY

The calculation and its inputs are present: **the answer can be reconstructed.**

UNDERSTAND · CHECKING ACCESS IS NOT DECODING

The code opens the gate.
Another calculation reads the content.

THE TWO FINGERPRINTS

`56ea3ff6`

`cd1014a9`

Same inputs,
two uses.

01 **Check access**

Calculate the expected code.

`B80FEF82`

02 **Decode the content**

Generate a sequence of numbers.

• • • …

31 BYTES FROM THE DUMP

Encrypted data

with

CALCULATED SEQUENCE

One number per byte

→

BITWISE COMBINATION

`BA26{…}`

THE TAKEAWAY

The solver recreates **the check, then decodes 31 bytes.**

THE EVIDENCE · THE DECODED RECORD

Follow the right branch.
Read the real record.

RECOVERED CODE

B80FEF82 ✓

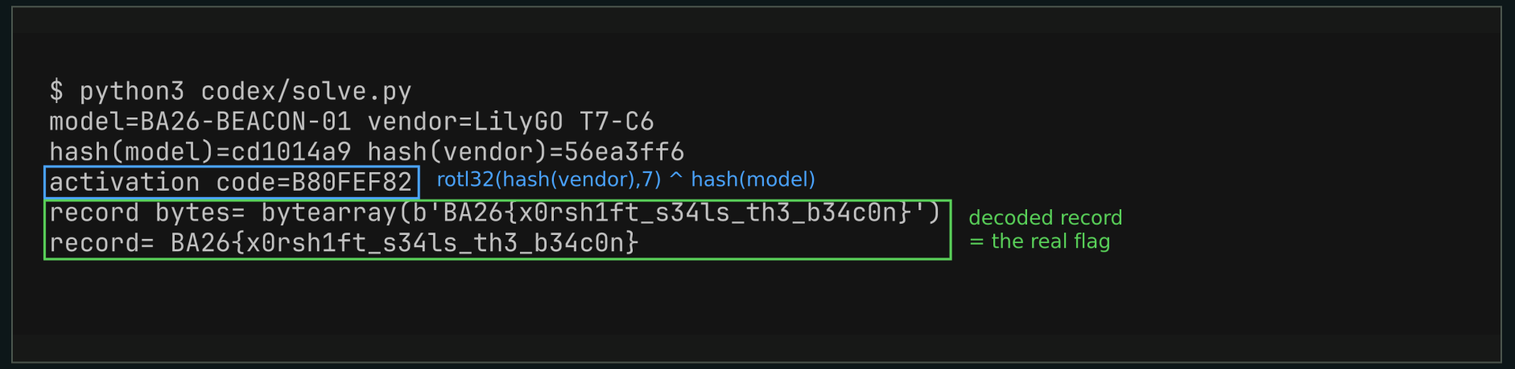
Code calculation

↓

Data decoding

↓

Readable record



SAVED EVIDENCE · CLICK TO ENLARGE

Solved by: **Codex** · Offline solver result.

`BA26{x0rsh1ft_s34ls_th3_b34c0n}`

THE TAKEAWAY

The evidence comes from the **decoding path**, not a string found by chance.